```
===== mlass6-1.jpg =====
import random
from collections import defaultdict
import tqdm

WIN_LINES = [
(0,4,8), (2,4,6)

def initial_board():
return " " * 9

def available_actions(state):
return [i for i, ch in enumerate(state) if ch == " "J

def apply_action(state, action, player):
s = list(state)
s[action] = player
return "".join(s)

def check_winner (state):
for a, b, c in WIN_LINES:
if state[a] == state[b] == state[c] and state[a] != " ":
return state[a]

if "" not in state:
return "D"

return None

class QAgent:
def __ init__(self, player="X", alpha=0.5, gamma=0.9,
epsilon=1.0, eps_min=0.1, eps decay=0.9999):

self.player = player
self.Q = defaultdict (float)
self.alpha = alpha
self.gamma = gamma
self.epsilon = epsilon
self.eps_min = eps_min
self.eps_decay = eps decay

def get_action(self, state, actions, greedy=False):
if not greedy and random.random() < self.epsilon:
```

```
return random. choice(actions )
qs = [self.Q[(state, a)] for a in actions]
maxq = max(qs)
best = [a for a in actions if self.Q[(state, a)] == maxq]

return random.choice(best)

def update(self, s, a, r, s_next, actions_next, done):

===== mlass6-2.jpg =====
q = self.Q[(s, a)]

if done or not actions_next:
target iF
else:
target = r + self.gamma * max(
self.Q[(s_next, an)] for an in actions_next

)
self.Q[(s, a)] = q + self.alpha * (target - q)
if self.epsilon > self.eps_min:

self.epsilon = max(
self.eps_min, self.epsilon * self.eps_ decay

def play_episode(agent, training=True):
state = initial_board()

player = "X"
winner = None
history = []
while True:

actions = available_actions(state)

if player == agent.player:
action = agent.get_action(state, actions)
next_state = apply_action(state, action, player)
winner = check_winner(next_state)

history.append((state, action, next_state, winner is not None))

else:
```

```
action = random.choice(actions)
next_state = apply_action(state, action, player)
winner = check_winner(next_state)

state = next_state

if winner is not None:
break

player = "0" if player == "X" else "X"

if training:
reward = 1 if winner == agent.player else @.5 if winner == "D" else -1

for i, (s, a, s_next, done) in enumerate(history):
next_actions = available_actions(s_next)
is_last = (i == len(history) - 1)
r = reward if is_last else @

agent .update(
S, a, r, S_next, next_actions, done and is_last

)

===== mlass6-3.jpg =====
return winner

def train(agent, episodes=2000@) :
results =) {"X": @, "0": @, “D": @}

for _ in tqdm.trange(episodes):
winner = play_episode(agent)
results[winner] += 1

return results

def evaluate(agent, games=10@0):
wins = losses = draws = @
old_epsilon = agent.epsilon
agent.epsilon = @

for _ in range(games):
winner = play_episode(agent, training=False)
```

```
if winner == agent.player:
wins += 1

elif winner == "D":
draws += 1

else:

losses += 1
agent.epsilon = old_epsilon

return {
"wins": wins,
"draws": draws,
"losses": losses

agent = QAgent()

print("Training...")
train(agent, episodes=30000)

print("Testing...")
print(evaluate(agent, games=1000) )

Training...

1007 | ee | 3000
@/30000 [00:01<00:00, 25881.07it/s]

Testing...

{'wins': 874, ‘draws': 104, 'losses': 22}

def show_board(state):
print(
f"{state[@]}|{state[1]}|{state[2]}\n"
"-4-4-\n"
#"{state[3]}|{state[4]}|{state[5]}\n"
"-4+-4-\n"

===== mlass6-4.jpg =====
#"{state[6]}|{state[7]}|{state[3]}"
```

```
def play_demo(agent):
state = initial_board()
player = "X"

show_board(state)
print()

while True:
actions = available _actions(state)

if player == agent.player:
action = agent.get_action(state, actions, greedy=True)
print(f"Agent plays position {action}")
else:
action = random.choice(actions)
print(f"Opponent plays position {action}")

state = apply _action(state, action, player)
show_board(state)
print()
winner = check_winner(state)
if winner is not None:
print("Winner:", winner)
break
player = "0" if player == "X" else "X"

for _ in range(3):
play_demo(agent)
```

===== mlass6-5.jpg =====

```
Agent plays position 1

Opponent plays position 2
|x|o

-+-4-

| |

-4-4-

Agent plays position 7
```

[x|0

-+-+-
| |

apa ge
Ix|

Opponent plays position 5
Ix|0

eleedls

| Jo

-+-4-

x|

Agent plays position 4
|x|0

wepengee
[x|0

-+-+-
Ix|

Winner: X

-+-+-

apoqis

Agent plays position 1
Ix|
-+-+-

Opponent plays position 2

===== mlass6-6.jpg =====
Agent plays position 5

Opponent plays position 7
|x|o
-+-4-

Agent plays position 3

Opponent plays position 4
Ix|0

eleedls

X|O|X

-+-4-
|0|

Agent plays position 6
|x|0

apewe

x|0|X

-+-+-

x|O|

Opponent plays position @
0|x|o

-+-+-

x|0|X

wieedis

x|0|

Agent plays position 8
0|x|o
-+-+-
X|0|X
=pewe

x|0|X

Winner: D

===== mlass6-7.jpg =====
Agent plays position 2
| [x
-}-4-

Opponent plays position 7
| |x
-+-4-
| |
-4-4-
|0|

Agent plays position 3
| Ix
-+-+-
x] |
<page
|0|

Opponent plays position @
o| |x
leeds
x| |
-+-4-
|0|

Agent plays position 6
o| |x
apewe
x] |
-+-+-
X|O|

Opponent plays position 5
o| |x
-+-+-
X| |o
weeds
x|0|

Agent plays position 4
o| |x

-+-+-

X|xX|0

=pewe

X|O|

Winner: X

===== mlass6-8.jpg =====